The proposed MVP should learn reusable product facts as complaints progress, ask the user when evidence is missing, and resume from saved progress. We can do this with a small workflow: product JSON first, supporting documents second, user input when needed.

Below I've addressed every annotation. These are design decisions and recommendations; the application still uses the existing scripted flow.

1. "Typed intermediate outputs" means predictable information passed between stages.

Suppose the business asks:

> "And when did you buy it?"

The resolver could return this internal result:

```
{
  "originalQuestion": "And when did you buy it?",
  "standaloneQuestion": "When was the product in this complaint purchased?",
  "requestedFacts": ["purchase.date"],
  "needsClarification": false
}
```

The retrieval stage could return:

```
{
  "status": "found",
  "facts": {
    "purchase.date": "2026-02-12"
  },
  "source": {
    "type": "product_knowledge",
    "revision": 3
  }
}
```

"Typed" means we define which fields and values are permitted. For example, retrieval status might be `found`, `missing`, `conflicting`, or `unavailable`.

This lets the backend distinguish "ask the user" from "generate an answer." It also helps diagnose which stage went wrong. **Valid JSON alone does not prove the interpretation or retrieved fact is correct.**

2. Validation before actions means checking a proposed booking against actual business state.

Example:

- User's preference: **20 October, 10:00–12:00**.
- Business offers that appointment with identifier `slot-42`.
- Custra presents it.
- User selects **Confirm visit**.
- Backend checks that `slot-42` belongs to this complaint, matches the confirmation, is eligible for booking, and has not already been booked.

- Backend sends the booking request.
- Custra displays **Confirmed** after receiving the business confirmation.

If the model says “Your visit is confirmed” before that response, the backend should prevent that claim from becoming the displayed booking status.

The model helps communicate; the backend controls the booking transition.

3. The explicit workflow is simply a small set of named stages and branches.

My recommendation is:

```
Receive question
    → Resolve meaning
    → Find evidence
        → Enough evidence: compose answer
        → Missing information: ask user
        → Conflicting information: ask user to resolve it
        → Search unavailable: explain limitation and ask user
    → Send supported answer
    → Save progress
```

Appointment offers enter a separate confirmation-and-booking branch.

We can implement this using ordinary backend functions and explicit state. **A graph framework or multiple deployed agents is unnecessary for this MVP.**

4. User handoff should fill missing facts and improve future complaints.

Agreed. When Custra cannot answer:

> **Business:** “What is the serial number?”
> **Custra to user:** “I couldn’t find the serial number. Please enter it or attach a photo of the product label.”
> **User:** “SN-88421.”

Custra should:

1. Save the user’s response in the complaint history.
2. Associate it with the pending business question.
3. Save the reusable serial-number fact under that product, recording its source as the user’s response.
4. Answer the business.
5. Make the fact available to later complaints.

We should distinguish what can be reused:

User supplies	Where it belongs	Reuse
Serial number, model, purchase date	Product knowledge	Across complaints for that product
Corrected phone number	Current complaint; update saved profile when the user indicates a lasting change	According to that scope

User supplies	Where it belongs	Reuse
“Use my office address for this visit”	Current complaint	Not automatically the default for future visits
“It started leaking yesterday”	Current complaint	Historical evidence, not the next complaint’s current issue
“Tuesday morning works”	Current appointment preference	Not a permanent preference

This is how knowledge improves over time without turning every chat statement into a permanent fact.

A user correction should become effective immediately for **subsequent answers**. It should not silently alter messages already sent.

5. Versioning preserves what was known when an answer was sent.

Example:

Time	Product knowledge	Conversation
Complaint starts	Revision 3: purchase date is 12 February	Business asks for the date
Custra answers	Revision 3	“12 February 2026”
User corrects it	Revision 4: purchase date is 15 February	“The saved date is wrong; it was the 15th.”
Custra continues	Revision 4	Sends the correction to the business

The old answer remains in history, associated with revision 3. The correction and future answers use revision 4.

For MVP, we can keep this small: a current knowledge revision, source references, and a record of changed facts. We do not need to copy the entire product document into a new archive after every message.

6. Image content determines its role; the file extension does not.

A JPEG can contain an invoice, warranty certificate, serial-number label, product photo, or unrelated content.

For MVP, the multimodal LLM should inspect the attachment and return:

- Document/media category.
- Whether it appears relevant to the selected product or pending question.
- Extracted facts and supporting text.
- Any uncertainty or conflict.

Examples:

Uploaded content	Treatment
Photograph of an invoice	Extract invoice and relevant purchase details

Uploaded content	Treatment
Extended-warranty certificate image	Extract recorded warranty details
Serial-number label	Extract exact model/serial characters
Ordinary appliance photo	Retain as complaint evidence; extract identifiers only if visible
Unrelated image	Do not populate product facts
Unclear image	Ask the user what it contains or request the missing detail

The LLM's classification can be wrong. Unreadable characters, product mismatches, and conflicting values should trigger clarification. A model-generated confidence number alone should not decide whether a fact is safe to save.

7. Populate JSON first, then inspect documents for facts the JSON missed.

Yes—that is the approach I recommend, incorporating your agreement:

- Automatically populate clear, nonconflicting extracted facts with source references.
- Ask for review when values are uncertain or conflict.
- Preserve the original attachment.
- Consult supporting material when a later question requires something not yet captured.

For example:

> Invoice extraction saves purchase date and invoice number.
> Later, the business asks: "Which branch issued the invoice?"

If the branch was not extracted initially, Custra can inspect the invoice with that precise question. A supported answer can then be saved as another reusable product fact.

JSON extraction need not anticipate every future question. The original evidence remains available.

8. Use multimodal LLM extraction for MVP; evaluate specialized extraction later.

Agreed. A supported multimodal Gemini model can inspect PDFs/images and produce structured extraction results. Google documents PDF understanding and extraction through Gemini; we still need to verify the chosen model's capabilities during implementation. Google document-understanding documentation

For a small product-specific collection, I recommend starting with **bounded, question-directed inspection of relevant attachments**, rather than requiring a vector index for every file.

Vector search remains useful if the document collection or passage volume makes direct inspection inefficient. This is a proposed simplification to the earlier vector-first design, subject to evaluation.

I recorded specialized OCR/layout/entity-extraction evaluation in future development. We should compare accuracy, latency, and cost before replacing the LLM approach.

9. Attachments during registration must participate in the pending conversation.

Understood—you mean media uploaded while the user is talking to the business through Custra.

For example:

> **Business:** “Please provide the invoice number.”
> **User uploads:** A photograph of the bill.

Custra should attach that file to the current complaint and pending question, inspect it, extract the invoice number, and continue with the supported answer.

Reusable product facts extracted from the file can enter product knowledge. The original upload should retain its complaint association. This makes its origin clear while allowing relevant evidence to help future complaints about the same product.

Uploading a file to Custra does **not automatically mean sending the original file to the business**. If the business requests the file itself, that requires an explicit supported attachment-transfer path.

10. Deferred scope is recorded.

Added to future development:

- Troubleshooting, diagnosis, repair advice, and broad warranty interpretation.
- Service-center selection and appointment booking.

The proposed MVP focuses on technician-visit registration and confirmation.

11. Resolver behavior and the number of LLM calls.

Understood: your point is resolving follow-up questions using previous questions and answers. We will retain clarification as a general fallback when meaning is genuinely uncertain.

And **yes, one LLM call can perform several closely related interpretation tasks**:

- Rewrite the question into a standalone form.
- Identify the information requested.
- Identify relevant entities or constraints.
- Indicate whether clarification is needed.

Retrieval happens after that interpretation. Answer generation will generally be another call because it consumes the retrieved evidence.

A reasonable starting flow is:

```
Call 1: Interpret question and select required information
Backend: Retrieve evidence
Call 2: Compose answer using evidence
```

An attachment-processing call is additional when a document needs inspection. We do not need a separate model call for every individual label in the workflow.

12. Resolved questions stay internal; feedback comes later.

Agreed. The customer should see the actual business question and Custra’s response.

The standalone rewrite, retrieval decisions, and evidence references are internal diagnostic records.

Thumbs-up/down feedback and controlled retry of an evaluated step are recorded for later development. Feedback should be linked to a particular turn so we can inspect whether the error arose in interpretation, retrieval, or answering.

13. App closure should lead to recovery from saved state.

Your proposed **Retry** experience is appropriate, with one adjustment: closing the app does not prove the request failed. The backend or business may have completed it.

When the user returns:

Saved outcome	Behavior
Step completed	Display its result and continue
Request still owns an active operation	Show pending/checking status
Step failed or can safely resume	Offer **Retry saved step**
Booking outcome is uncertain	Recover the outcome using the existing operation identity before attempting another booking

The retry resumes the same complaint and pending operation. It should not recreate the complaint or blindly submit a second booking.

No background conversation worker is needed for this experience. An already-dispatched request may finish; subsequent steps can wait for the user’s return.

14. “Scoped searchable passages” and failure handling.

A passage is a portion of extracted document content. “Scoped” means a search is restricted to relevant, permitted evidence.

For example, an invoice passage could carry:

```
Product: product-123
Document: invoice-456
Version: 1
Page: 1
Content: “Invoice number INV-123; purchase date …”
```

A complaint for product-123 should not accidentally retrieve a similar invoice from another product or inaccessible workspace.

For MVP, honor your request for **no new automatic retry mechanism**:

Failure	Immediate behavior
Extraction fails	Keep the file; leave facts unpopulated; ask the user for the needed detail
Embedding fails, if indexing is introduced	Keep extracted facts/file; mark semantic search unavailable
Retrieval service fails	Distinguish unavailable search from missing information; ask the user
Retrieval returns no supporting evidence	Ask the user
User supplies the fact	Save it with provenance and continue

The planned fallback from JSON to document inspection is an evidence source choice, not an automatic retry of a failed request.

Conditional automatic retries are deferred. **User-initiated resume after reopening remains a separate feature.**

15. Should all conversations for a product live in one document?

I recommend **one logical product history, implemented using separate complaint records**, while retaining the existing complaint event storage.

```
Product
├── Current reusable knowledge
├── Supporting-document references
└── Linked complaints
    ├── Complaint A → messages and outcomes
    └── Complaint B → messages and outcomes
```

This gives the product a complete history without merging every conversation into one growing record.

Reasons:

- Existing complaint/event storage already supports it.
- Different complaints have different symptoms, addresses, and appointment decisions.
- Separate records make reopening and updating a complaint straightforward.
- A large combined document would require repeatedly reading and updating unrelated history.

For the MVP, **promote reusable user-confirmed facts into product knowledge as they are learned**. That usually removes the need to search old conversations for serial numbers or purchase details.

I would defer general semantic search across past chats. If a question specifically requires historical information—“What was the previous complaint reference?”—we can retrieve the relevant complaint record directly. Old agent-generated statements should not automatically become trusted evidence.

This provides learning through saved knowledge, without adding a separate memory agent or vectorizing all chat history.

16. Example successful complaint journeys

These are fictional examples of the proposed behavior. The wording and answers must not become runtime question-to-answer mappings.

Scenario A — all required facts are already saved

User reports: “My washing machine is leaking.” Preferred visit: 20 October, 10:00–12:00.

Speaker	Exact message/action
Business	“Which appliance and model need servicing?”
Custra	“An Example Brand WM-100 washing machine.”
Business	“What issue are you reporting?”
Custra	“The washing machine is leaking.”
Business	“Where should the technician visit, and which number should we call?”
Custra	“10 Fiction Street, Pune 411001. Contact number: 9000000000.”
Business	“We can offer 20 October, 10:00–12:00. Would you like to book it?”
Custra → user	“Confirm the technician visit for 20 October, 10:00–12:00?”
User	Selects **Confirm visit**
Business	“Booking confirmed. Reference BOOK-101.”
Custra	Displays the confirmed visit and reference

Scenario B — follow-up question requires invoice inspection

Product model is known; purchase date is absent from JSON. An invoice is attached.

Speaker	Exact message/action
Business	“What model is the refrigerator?”
Custra	“Example Brand RF-200.”
Business	“And when did you buy it?”
Internal resolver	Resolves the question to the purchase date of the refrigerator in this complaint
Internal retrieval	Inspects its invoice; finds 12 February 2026; records the evidence
Custra	“It was purchased on 12 February 2026.”
Business	“What date and time would you prefer for a technician visit?”
Custra	“20 October, 10:00–12:00.”
Business	“That slot is available. Shall I reserve it?”
User	Confirms the displayed offer

Speaker	Exact message/action
Business	Returns the booking confirmation

Scenario C — missing fact supplied by the user, then reused

Speaker	Exact message/action
Business	“Could you provide the appliance’s serial number?”
Custra → user	“I couldn’t find the serial number. Please enter it or attach a photo of the label.”
User	“SN-88421.”
Internal update	Saves serial number with the user-message source
Custra → business	“The serial number is SN-88421.”
Business	“Can we arrange a visit on 20 October, 10:00–12:00?”
User	Confirms
Business	Returns the booking confirmation

In a later complaint for that product:

> **Business:** “What is the unit’s serial number?”
> **Custra:** “SN-88421.”

Custra retrieves the saved fact without asking again.

Scenario D — invoice photograph uploaded during chat

Speaker	Exact message/action
Business	“What is the invoice number for this product?”
Custra → user	“I couldn’t find the invoice number. Enter it or attach the invoice.”
User	Uploads `bill-photo.jpg`
Internal processing	Recognizes an invoice, associates its product line, extracts `INV-123`, and saves its source
Custra → business	“The invoice number is INV-123.”
Business	“What service address should we use?”
Custra	Supplies the current complaint’s selected address
Business	Offers the preferred visit
User	Confirms
Business	Returns the booking confirmation

Scenario E — user corrects a fact during the conversation

Speaker	Exact message/action
Business	“Please confirm the purchase date.”
Custra	“The recorded purchase date is 12 February 2026.”
User	“That is incorrect. It was 15 February 2026.”
Internal update	Saves the correction and increments the knowledge revision
Custra → business	“Correction: the customer confirms the purchase date was 15 February 2026.”
Business	“Thank you. Can we book the preferred technician slot?”
User	Confirms
Business	Returns the booking confirmation

The earlier answer remains visible, followed by its correction. Future complaints use the corrected date.